

Editable Python Package Setup for gpu_corrca1

This document explains how to structure a Python project so that it can be installed into a Conda environment using **pip install -e ..**. The goal is to avoid sys.path hacks and allow imports such as:

```
from gpu_corrca1.utils import something
```

1. Recommended Project Structure

```
gpu_corrca1/
├── pyproject.toml
├── README.md
├── src/
│   ├── gpu_corrca1/
│   │   ├── __init__.py
│   │   ├── utils/
│   │   ├── benchmarking/
│   │   └── simulation/
│   ├── experiments/
│   ├── scripts/
│   ├── tests/
│   └── cpp/
```

2. Purpose of Important Components

Component	Purpose
src/	Contains the actual Python package code.
gpu_corrca1/	Top-level Python package name.
__init__.py	Marks folders as Python packages.
pyproject.toml	Defines package metadata and installation behavior.
cpp/	CUDA/C++ backend code compiled into shared libraries.
scripts/	Runnable terminal entry-point scripts.
experiments/	Prototype and benchmarking work.

3. Minimal pyproject.toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "gpu_corrca1"
version = "0.1.0"
description = "GPU-accelerated correlation calibration tools"
requires-python = ">=3.9"

dependencies = [
    "numpy",
    "scipy",
]

[tool.setuptools.packages.find]
where = ["src"]
```

4. Minimal `__init__.py`

```
# src/gpu_corrca1/__init__.py
```

An empty `__init__.py` file is sufficient initially. Its main purpose is to mark the directory as a Python package.

5. Installing the Package

```
# From the repository root:  
pip install -e .
```

The '-e' flag means 'editable'. The environment points directly to the local source files, so edits immediately appear without reinstalling the package.

6. Verifying the Installation

```
python  
  
>>> import gpu_corrca1  
>>> print(gpu_corrca1)
```

7. Why This Matters

Using editable installs avoids: `sys.path.append(...)` hacks running scripts only from specific directories messy relative import behavior It also makes the repository behave like a real reusable Python package.

8. Typical Workflow

1. Create/update Python files under `src/gpu_corrca1/`
2. Run:

```
pip install -e .
```
3. Import package from anywhere:

```
from gpu_corrca1.utils import something
```
4. Modify files freely without reinstalling